Hoofdstuk 5

Het fysieke datawarehouse

In het vorige hoofdstuk heb je geleerd over de logische tabelstructuur van een datawarehouse. De verschillen in benadering en structuur tussen een dimensioneel model, een genormaliseerd model en een Data Vault zijn besproken. In dit hoofdstuk ga je meer kijken naar de fysieke implementatie. Welke software kun je gebruiken? Wat voor soort hardware heb je nodig? Hoe zorgt je ervoor dat de performance acceptabel zal zijn? Hoe houd je in de gaten wat de performance is? Allemaal zaken om over na te denken om een succesvolle implementatie van een datawarehouse te doen.

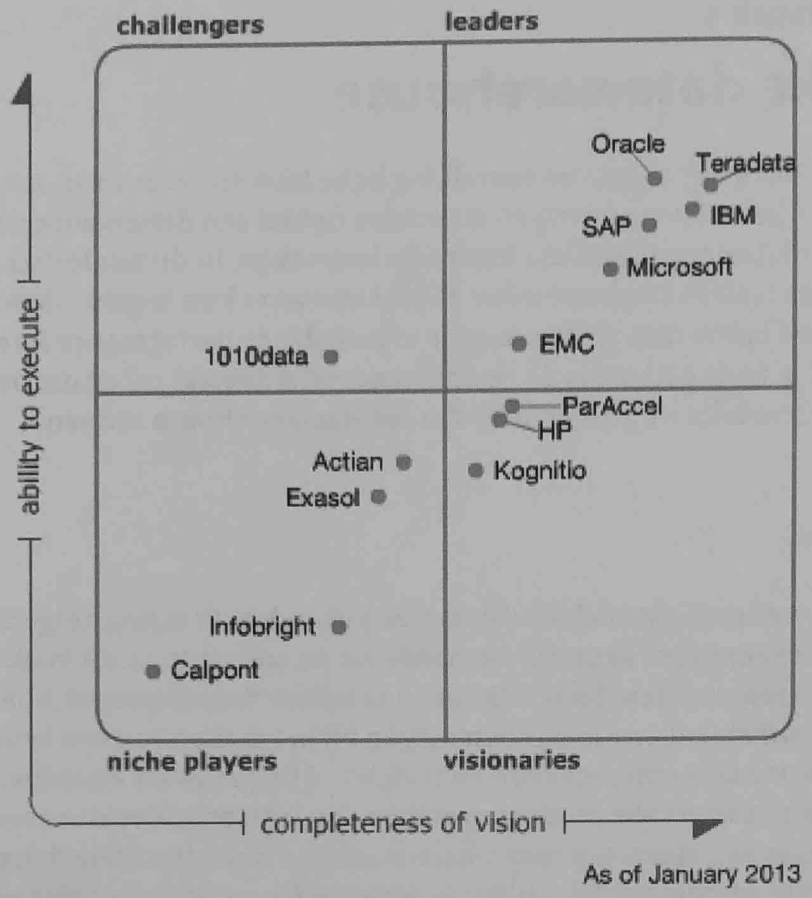
5.1 Inleiding

Tot nu toe hebben we het in dit boek heel generiek gehouden. Er is nog nergens gesproken over specifieke keuzes ten aanzien van hardware en software. In dit boek zal ook niet een keuze gemaakt worden. Toch is het goed te kijken naar de mogelijkheden op dit gebied en naar enkele zaken die kunnen helpen bij het maken van een keuze.

Elke keuze die je maakt komt met voor- en nadelen. Alle hardware en software heeft zo zijn beperkingen. Hoewel alle producten anders zijn, zijn er ook veel overeenkomsten. Daarmee zijn er ook algemeen toepasbare manieren om bijvoorbeeld performance van query's te verbeteren. We zullen in paragraaf 5.4 van dit hoofdstuk enkele manieren bekijken. Om concrete voorbeelden te kunnen geven maken we in dit boek gebruik van Microsoft SQL Server 2016. Je kunt de voorbeelden meedoen als je SQL Server installeert. Een gratis evaluatie-editie is te downloaden van de Microsoftsite (*www.microsoft.com/sqlserver*).

Vroeg of laat zal je tijdens een Business Intelligence-project een keuze moeten maken voor de te gebruiken hardware en software. Welk DBMS (Database ManagementSysteem) ga je gebruiken? Wordt het een op Microsoft SQL Server gebaseerd datawarehouse, moet het datawarehouse in Oracle gebouwd worden, kies je voor Teradata of wordt het een van de vele andere spelers op deze markt? Soms moet deze keuze nog echt gemaakt worden, vaak zal je geconfronteerd worden met een bestaande situatie. Niet altijd is de keuze voor hardware en software nog vrij. Het kan goed zijn dat er een eis is dat het datawarehouse gebruikmaakt van de bestaande, aanwezige hardware. Ook voor de software kan al lang een keuze zijn gemaakt, al was het maar omdat alle al bestaande databases op basis van hetzelfde DBMS zijn gemaakt.

Als je nog helemaal vrij bent zou Gartner (*www.gartner.com*) een startplek kunnen zijn om eens te kijken wat er zoal op de markt is. Zij publiceren elk jaar het datawarehouse magic quadrant. Figuur 5.1 toont het magic quadrant van 2017 voor datawarehousing. De producten van (bekende) leveranciers worden ingedeeld in de categorieën niche player, visionary, challenger en leader op basis van de compleetheid van het product, de visie rond het product en of ze daadwerkelijk het beloofde leveren.



Figuur 5.1 *Gartner magic quadrant*

Gartner stelt overigens op vele gebieden zo'n magic quadrant op. Zo is er bijvoorbeeld ook een over Analytics. Aangezien het datawarehouse maar een klein onderdeel is van de totale Business Intelligence-oplossing is het zeker interessant ook de conclusies van andere relevante magic quadranten te bekijken. Hou bijvoorbeeld de BI-roadmap in de gaten zoals besproken in hoofdstuk 2 en zoek de relevante quadranten op aansluitend op de uitgesproken ambities in de roadmap.

5.2 Technisch ontwerp

Voordat je iets zinvols kunt zeggen over welke hardware en software je nodig hebt voor een succesvolle implementatie, moet je eerst meer weten over je datawarehouse. Hoe groot gaat het datawarehouse worden? Welke eisen worden er gesteld aan het datawarehouse in termen van beschikbaarheid, data latency en performance? Hoeveel gebruikers zijn er en hoe complex is de workload?

Een goed punt om mee te beginnen is het kwantificeren van het logische model dat je hebt overgehouden aan hoofdstuk 4. De tabelstructuur die een gevolg is van je ontwerp (sterschema of anderszins) wordt wel het functionele of soms het logische ontwerp genoemd. Van dit functionele ontwerp moet nu een technisch ontwerp worden gemaakt.

Het **functionele ontwerp** van een database is de gewenste tabelstructuur, alleen rekening houdend met de functionele eisen zoals die naar voren komen uit de informatieanalyse.

Het doel van het maken van een technisch ontwerp is om het functionele model aan te passen aan de (niet ideale) werkelijkheid. Er kan zich een tweetal problemen voordoen:
- De performance is niet toereikend.
- De structuur is niet werkbaar gezien beperkingen van het gebruikte DBMS.

Om iets zinvols te zeggen over deze mogelijke problemen moet je weten hoe groot de verschillende tabellen gaan worden. De grootte van een tabel wordt bepaald door:
- het aantal records in een tabel;
- de gemiddelde recordlengte.

De gemiddelde recordlengte hangt uiteraard af van de inhoud van het datawarehouse (lengte van namen en omschrijvingen), maar ook van het datatype van de kolommen. In onderstaande paragrafen worden beide punten verder verklaard.

Het **technisch ontwerp** van een database past het functionele ontwerp zodanig aan dat de database in de praktijk goed functioneert.

5.2.1 Aantal records

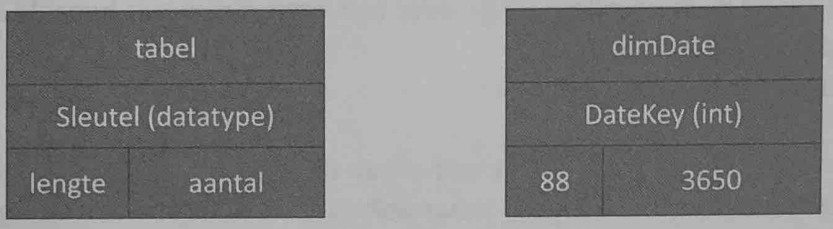
Als eerste moet je een inschatting proberen te maken van het aantal records dat je per tabel verwacht. Voor sommige tabellen is dat gemakkelijk in te schatten, voor andere is het moeilijker. Een datumdimensie is bijvoorbeeld rechttoe rechtaan. Als het datawarehouse gegevens bevat vanaf 1 januari 2010 en je de datumdimensie vult met gegevens tot 2020, heb je tien jaar aan records. Nu is het nog een kwestie van de grain weten. Met een grain op dagniveau, krijg je een tabel met 3650 records. Niet echt spannend.

Het spannendst zijn waarschijnlijk de feitentabellen. Dit zijn namelijk over het algemeen de grote tabellen. Als je bijvoorbeeld gespreksduur van gesprekken naar een callcenter opslaat, en je krijgt gemiddeld duizend gesprekken per dag, hebben we het al over 365.000 records per jaar. Met tien jaar historie is dit meer dan 3,5 miljoen records. In een modern datawarehouse is dat nog een kleine tabel. Sommige feitentabellen bevatten miljarden records.

Het belangrijkste hier is dat je in orde van grootte probeert een inschatting te maken van het aantal records, zowel nu als in de toekomst. Als je ziet aankomen dat een tabel erg groot gaat worden, moet je vooraf nadenken over hoe die omvang te beheren. Wellicht dat partitionering van de tabel zinvol is. Later in dit hoofdstuk zie je een voorbeeld van tabelpartitionering. Vooraf strategieën bedenken en implementeren om grote tabellen beheersbaar te houden is niet moeilijk. Achteraf dezelfde oplossing implementeren zou wel eens erg duur kunnen zijn. Beter voorkomen dan genezen, dus!

Sommige tabellen in datawarehouses worden zo groot dat ze onbeheersbaar worden. Als je dat vooraf ziet aankomen kun je maatregelen treffen.

Je kunt een databasediagram uitbreiden met de bedachte aantallen. Een tabel wordt dan weergegeven zoals in figuur 5.2. Je ziet de naam van de tabel en de primaire sleutel. In het voorbeeld zie je de tabel dimDate met de sleutel DateKey. Tussen haakjes staat het datatype van de sleutel. Op de onderste regel zie je links de gemiddelde recordlengte in bytes en rechts het aantal te verwachten records. Vermenigvuldigd levert dit de grootte van de tabel in bytes op.



Figuur 5.2 *Kwantificeren*

5.2.2 Gemiddelde recordlengte

Naast het aantal te verwachten records is de lengte van de records belangrijk. Om dat te bepalen moet je drie zaken weten:
- Uit welke kolommen bestaat de tabel?
- Wat is het datatype van elke kolom?
- Wat is de gemiddelde lengte van bijvoorbeeld namen?

Uit het functionele model dat het resultaat is van wat je in hoofdstuk 4 hebt geleerd, kun je direct aflezen uit welke kolommen een tabel bestaat.

De volgende stap is te specificeren welk datatype de kolommen gaan krijgen. Daarvoor moet je weten welk DBMS je gaat gebruiken. Alle databases hebben vergelijkbare datatypes. Vergelijkbaar betekent echter niet hetzelfde.

Het **datatype** van een kolom bepaalt het soort gegevens dat in deze kolom opgeslagen kan worden, bijvoorbeeld een datum of een getal.

Grofweg heeft een database drie datatypes:
- numeriek;
- alfanumeriek;
- datum.

Dit lijstje kan worden aangevuld met grote datatypes. Hierbij moet je denken aan afbeeldingen, XML, grote tekstvelden en hele documenten.

Numeriek
Numerieke datatypes kunnen worden onderverdeeld in gehele en gebroken getallen. SQL Server kent bijvoorbeeld vier soorten gehele getallen, de tinyint, de smallint, de int en de bigint. Deze zijn respectievelijk 1, 2, 4 en 8 byte groot. Hoe groter het datatype, hoe groter de getallen die je kunt opslaan, maar ook hoe meer opslagcapaciteit en geheugen je nodig hebt. Een tinyint kan alleen positieve getallen opslaan met een maximum van 255. Dat is meer dan genoeg voor een kolom DagNummerVanWeek (1 t/m 7), DagNummervanMaand (1 t/m 31), maar al te klein voor DagNummerVanJaar (1 t/m 366). De andere integer datatypes kunnen zowel positieve als negatieve getallen opslaan met als maximum 2 tot de macht (n-1) met n het aantal bytes maal 8. Voor de bigint is dat 9223372036854775807.

■ Hoe groter het datatype, hoe groter de getallen die je kunt opslaan, maar ook hoe meer opslagcapaciteit en geheugen je nodig hebt.

Gezien de hardware van tegenwoordig kun je je afvragen hoe belangrijk het is om je druk te maken over het verschil tussen 1 byte en 8 byte. Hoewel dat een terechte vraag is, gaat het nog altijd om de grote getallen. Stel dat een tabel een miljard records bevat. Het verschil is dan opeens zeven miljard byte, ofwel 7 gigabyte. En we hebben het hier nog maar over één kolom! Los van de storage die dat kost, houdt het ook inefficiënt geheugengebruik in. Dat kan leiden tot meer IO (interactie van de database met de harde schijf), wat kan leiden tot slechte performance. Hoewel het argument aanzienlijk minder belangrijk is dan enkele jaren geleden, blijft het de moeite waard om naar te kijken.

■ Kies altijd het kleinste datatype dat gegarandeerd groot genoeg is voor wat je met een kolom gaat doen.

Naast gehele getallen zijn er gebroken getallen. Gebroken getallen kunnen vaak nog worden onderverdeeld in exacte datatypes en benaderde datatypes. Voor benaderde datatypes geldt over het algemeen dat ze een groter bereik hebben, dat ze grotere getallen kunnen opslaan. Daar staan afrondingsproblemen in berekeningen tegenover. Je kunt zelf aangeven hoe groot de precisie van deze datatypes moet zijn, dus hoeveel posities achter de komma je wilt bijhouden. Hier geldt natuurlijk ook weer dat hoe groter het bereik en hoe groter de precisie, hoe meer storage en geheugen je nodig hebt.

Afhankelijk van het gebruikte DBMS zijn er nog andere numerieke datatypes. SQL Server kent bijvoorbeeld money en smallmoney, die respectievelijk 8 en 4 byte geheugen gebruiken en een precisie kennen van vier posities achter de komma.

Alfanumeriek
Alfanumerieke velden kennen over het algemeen een variant met vaste lengte en een variant met variabele lengte. Voor namen is de laatste variant het meest geschikt. Voor een postcodekolom kun je beter een vaste lengte kiezen.

Een variabele lengte datatype gebruikt evenveel opslagcapaciteit en geheugen als dat er nodig is. Je geeft alleen op wat de maximale lengte kan zijn. Per DBMS kan deze maximumlengte verschillen.

Belangrijk bij alfanumerieke velden is nog de code page die je gaat gebruiken. De code page bepaalt welke karakters de database ondersteunt. Het is belangrijk namen goed te kunnen opslaan, ook van mensen met een buitenlandse afkomst waarvan de namen karakters bevatten die niet in het Nederlandse alfabet voorkomen. Om zo flexibel mogelijk te zijn is het verstandig Unicode te gebruiken. De database gebruikt dan twee (of meer) bytes per karakter in plaats van één maar kan wel alle mogelijke karakters opslaan. De functionele eisen en flexibiliteit lijken het hier in de praktijk wel te winnen van de storage- en performance-eisen.

Datum

Voor het opslaan van datums bestaat over het algemeen een apart datatype. In sommige systemen wordt een datum altijd gecombineerd met een tijd. Dit kan van belang zijn bij het schrijven van de juiste query's.

Gemiddelde lengte van namen

Nadat je van alle kolommen het datatype hebt bepaalt, moet je nog de inhoud kennen. Het is niet genoeg om te weten dat een kolom ProductNaam het datatype `varchar(50)` heeft. Je moet ook weten uit hoeveel letters een productnaam gemiddeld bestaat.

Als je van elke kolom het datatype kent, en als je van alle kolommen met een variabele lengte de gemiddelde lengte kent, kun je de gemiddelde recordlengte uitrekenen. Maar waarom is dat belangrijk?

5.2.3 Page size

De meeste databases slaan records op pagina's op. Bij SQL Server is zo'n pagina altijd 8 kB groot. Na het aftrekken van wat overhead blijft er 8060 byte vrije ruimte over. Dat is om twee redenen belangrijk om te weten:

- Wat gebeurt er als de gemiddelde recordlengte groter is dan een pagina?
- Wat gebeurt er als het gemiddelde groter is dan een halve pagina?

Grote records

SQL Server slaat een record in principe op één pagina op. Je kunt niet twee vaste lengtekolommen opnemen in een tabel die beide bijvoorbeeld 5000 byte groot zijn. Alle vaste lengtekolommen samen moeten minder dan 8060 byte groot zijn. Een record kan wel groter worden dan 8060 byte, als dat maar komt door variabele kolommen. Alle gegevens die niet meer op de pagina passen, worden in een zogenaamd overflowgebied opgeslagen. Het ophalen van die gegevens is echter minder efficiënt dan het inlezen van een pagina. Je betaalt dus een prijs in termen van performance.

Bij dimensietabellen kan het bovenstaande een probleem worden. De theorie zegt dat je zoveel mogelijk beschrijvende kolommen moet opnemen in de dimensie. Daarmee loopt de gemiddelde recordlengte al snel op. Inefficiëntie en dus slechte performance ligt op de loer.

Met het toepassen van verticale partitionering of snowflakes kun je de problemen voorkomen. Je offert het ideale stermodel op ten behoeve van de performance. In paragraaf 5.4.5 zie je een voorbeeld van het effect van verticale partitionering.

- Het is belangrijk om in een vroeg stadium de gemiddelde recordlengte en page size te weten, zodat je een model kunt maken dat rekening houdt met beperkingen waardoor er later geen performanceproblemen zullen ontstaan.

Bij Oracle heb je een extra optie als records gemiddeld erg groot worden. Oracle slaat records op in blocks. Een block in Oracle komt overeen met een pagina in SQL Server. In Oracle kun je de grootte van een block zelf instellen. Je hoeft dus niet per se de tabelstructuur aan te passen, je kunt de database zo configureren dat hij goed aansluit bij de tabelstructuur.

- Je moet altijd kijken hoe het functionele ontwerp aansluit bij de eventuele beperkingen van het DBMS en aanpassingen doen waar dat nodig is.

Paginavulling

Stel je eens de volgende heel extreme situatie voor. Alle records zijn precies 1 byte groter dan een halve pagina. Het eerste record dat je opslaat komt op de eerste pagina te staan. Het tweede record past net niet meer op de eerste pagina en komt dus op een tweede pagina. En zo verder voor alle volgende records. Grofweg heb je een gigabyte aan storage nodig om 500 MB aan gegevens op te slaan. Ook het werkgeheugen van de computer zal op gelijke inefficiënte manier gebruikt worden. Los van storagekosten en het extra geheugen dat de server nodig heeft, zal dit resulteren in slechte performance.

Uiteraard zal het in de praktijk niet zo extreem zijn als hierboven beschreven. Het effect treedt echter wel degelijk op, inclusief de bijbehorende negatieve impact op de performance.

Samenvatting

Om zicht te krijgen op eventueel te verwachten performanceproblemen wil je weten hoe groot tabellen zijn. Om dat te berekenen moet je het aantal records kennen en de gemiddelde recordlengte. Voor de gemiddelde recordlengte moet je het datatype van alle kolommen weten en het gemiddeld aantal karakters dat datatypes met variabele lengte gaan bevatten.

Met deze informatie kun je het functionele ontwerp van het datawarehouse vertalen naar een technisch ontwerp. Het doel is om problemen met performance en beheersbaarheid van het datawarehouse voor te zijn.

5.2.4 Grootte van de database

Met de informatie uit de vorige paragrafen kun je de grootte van het datawarehouse berekenen. Er is echter nog een aantal factoren dat effect heeft op de totale omvang van de database. Je moet dan vooral denken aan:

- compressie van de gegevens;
- indexen.

Compressie

De meeste DMBS'en kennen tegenwoordig compressie van gegevens. Populaire programma's zoals Winzip en 7zip maken bestanden op je harde schijf kleiner. Dat is handig als je schijfruimte tekort komt of bij het uitwisselen van gegevens (downloaden van het internet, cloud storage zoals Dropbox, gebruik van kleine usb-flashdrives). Het is ook erg handig bij grote tabellen. Je bereikt namelijk twee dingen met compressie:

- je hebt minder opslagcapaciteit nodig;
- betere performance.

Minder opslagcapaciteit is gezien de kosten van opslag misschien niet heel erg spannend. Maar van kleinere databases kunnen ook sneller back-ups gemaakt worden. Daarmee wordt ook op de opslag van back-ups bespaard. Bovendien is er 's nachts meer tijd voor andere zaken (ETL, processen van kubussen) als de back-up korter duurt. Daarnaast kunnen kleine back-ups sneller gerestored worden. Restore-tijden kunnen belangrijk zijn voor afgesproken SLA's (Service Level Agreement). Een kleinere database heeft dus veel voordelen.

Vaak denken mensen dat compressie weliswaar goed is voor de benodigde opslagcapaciteit, maar dat het ten koste gaat van de performance. Performanceverbetering kan echter een belangrijke reden zijn compressie toe te passen. Bij query's waarvoor gegevens van de harde schijf gehaald moeten worden, is de performance van die schijf vaak de beperkende factor. Schijven zijn over het algemeen veel trager dan geheugen. Door het toepassen van compressie hoeven er minder gegevens direct van de schijf ingelezen worden. Daarmee gaan query's beter performen.

In paragraaf 5.4 van dit hoofdstuk kun je een voorbeeld van compressie zien.

Indexen

Naast compressie hebben vooral indexen impact op de grootte van de database. Een index neemt altijd extra ruimte in beslag, de database wordt dus groter. Een index maakt het als het goed is makkelijker om bij de juiste gegevens te komen. Dat zorgt voor een betere performance van query's.

In paragraaf 3.2.2 heb je kunnen lezen over indexen. Voor het inschatten van de grootte van de database is het van belang te weten welke indexen je gaat maken. Welke indexen nuttig zijn hangt volledig af van de query's die uitgevoerd gaan worden. In zijn algemeenheid kan een datawarehouse na het maken van indexen tweemaal zo groot worden.

In paragraaf 5.4 van dit hoofdstuk kun je een voorbeeld van indexing zien.

5.2.5 Overige factoren

Nu je enig zicht hebt op de grootte van de tabellen en de database als geheel, kun je gaan kijken naar de te verwachte performance van de database en de hardware die je daarvoor nodig hebt. Voordat we iets over hardware zeggen zullen we stil te staan bij:

- de te verwachte workload;
- het aantal gebruikers;
- beschikbaarheidseisen.

Workload

Om iets zinvols te kunnen zeggen over de performance moet je de workload kennen: welke query's zullen er uitgevoerd worden? Als je een goede informatieanalyse hebt gedaan, weet je dat. Welke rapporten moeten er gemaakt worden op het datawarehouse? Met welke frequentie en door hoeveel gebruikers wordt een rapport opgevraagd? Welke informatie staat op het rapport en tot welke query's leidt dat?

Het kan zijn dat alle rapporten gebaseerd zijn op een kubus. In dat geval is het processen van de kubus de enige workload op het relationele datawarehouse. Het andere uiterste is dat alle rapporten rechtstreeks hun gegevens door middel van complexe query's uit het relationele datawarehouse halen.

Om een inschatting te maken van de workload kun je query's indelen in simpele, gemiddelde en complexe query's:

- Simpele query's zijn gewone rechttoe rechtaan SELECT-statements, zonder moeilijke functies, berekeningen of constructies.
- Gemiddelde query's worden met een grote frequentie uitgevoerd (veelgebruikte rapporten) en bevatten een aantal joins en aggregaties.
- Moeilijke query's bevatten complexe logica. Soms moet een tabel vaker benaderd worden om de gewenste resultaten te berekenen. Een grote tabel benaderen zal altijd spannend zijn vanuit een performanceperspectief, als die tabel dan ook nog enkele keren benaderd moet worden om tot de juiste resultaten te komen, wordt de performance een uitdaging.

Aantal gebruikers

Het aantal zogenaamde concurrent gebruikers is medebepalend voor de belasting op je systeem. Concurrent betekent gelijktijdig. Hoeveel gebruikers zullen er tegelijkertijd gebruikmaken van het datawarehouse? Als alle query's heel eenvoudig zijn, maar er worden er veel tegelijkertijd uitgevoerd, zal het systeem veel intern geheugen nodig hebben, zeker als de datasets die uit die query's komen groot zijn. Met maar enkele gebruikers die veel complexe query's uitvoeren is CPU-kracht waarschijnlijk belangrijker.

Beschikbaarheid

Ten slotte moet je nagaan welke eisen een organisatie stelt aan de beschikbaarheid van het datawarehouse. Sommige datawarehouses zijn lokaal en worden alleen gebruikt tijdens lokale kantoortijden. Andere datawarehouses zijn corporate datawarehouses met gebruikers van vestigingen over de hele wereld. Er is dan altijd wel ergens een

kantoor open en dus wordt het datawarehouse 24 uur per dag gebruikt. Dit laatste scenario maakt bijvoorbeeld het laden van het datawarehouse een stuk lastiger.

Naast deze 'gewone' beschikbaarheid kunnen er nog eisen zijn met betrekking tot downtime. De servers zullen moeten worden 'gepatched'. Service packs moeten worden geïnstalleerd en dat gaat vaak samen met het herstarten van de machine. Kan dat wel in een 24/7-omgeving?

Los van de geplande downtime voor onderhoud kan er altijd sprake zijn van ongeplande downtime. Wat gebeurt er als een computer uitvalt? Wat gebeurt er als de storage-oplossing uitvalt? Wat gebeurt er als een heel datacenter uitvalt?

In termen van beschikbaarheid (high availability) zijn er veel mogelijkheden. Het is voornamelijk een kwestie van het afwegen van de risico's tegen de kosten. Wat is het risico dat er iets fout gaat, wat zijn de kosten als dat daadwerkelijk gebeurt, en wat zijn de kosten om ons tegen dit risico te beschermen?

Samenvatting

De moraal van dit hoofdstuk en van deze paragraaf is: weet vooraf wat de organisatie van het datawarehouse verwacht. Bekijk alle factoren die van invloed zijn op deze verwachtingen. Stem het ontwerp van het datawarehouse en de verwachtingen van de organisatie op elkaar af.

5.2.6 Grootte van je oplossing

Uiteraard is het heel moeilijk om uitspraken te doen over hoe groot en hoe complex een Business Intelligence-oplossing is. Met bovenstaande criteria zou een onderverdeling er als volgt uit kunnen zien:

	Klein	**Middelgroot**	**Groot**
Volume gegevens	<= 1 TB	1 t/m 10 TB	> 10 TB
Query complexiteit	>= 50% eenvoudig +- 30% gemiddeld < 10% complex	+- 50% eenvoudig 30 - 35% gemiddeld 10 - 15% complex	30 - 35% eenvoudig +- 40% gemiddeld 20 - 25% complex
Aantal gebruikers	<= 100 gebruikers, 10-20 concurrent	<= 1000 gebruikers, 100-200 concurrent	>1000 concurrent gebruikers,
Beschikbaarheid	Kantooruren	1 uur per nacht down	24 x 7 beschikbaar

Figuur 5.3 *Datawarehouse-classificering*

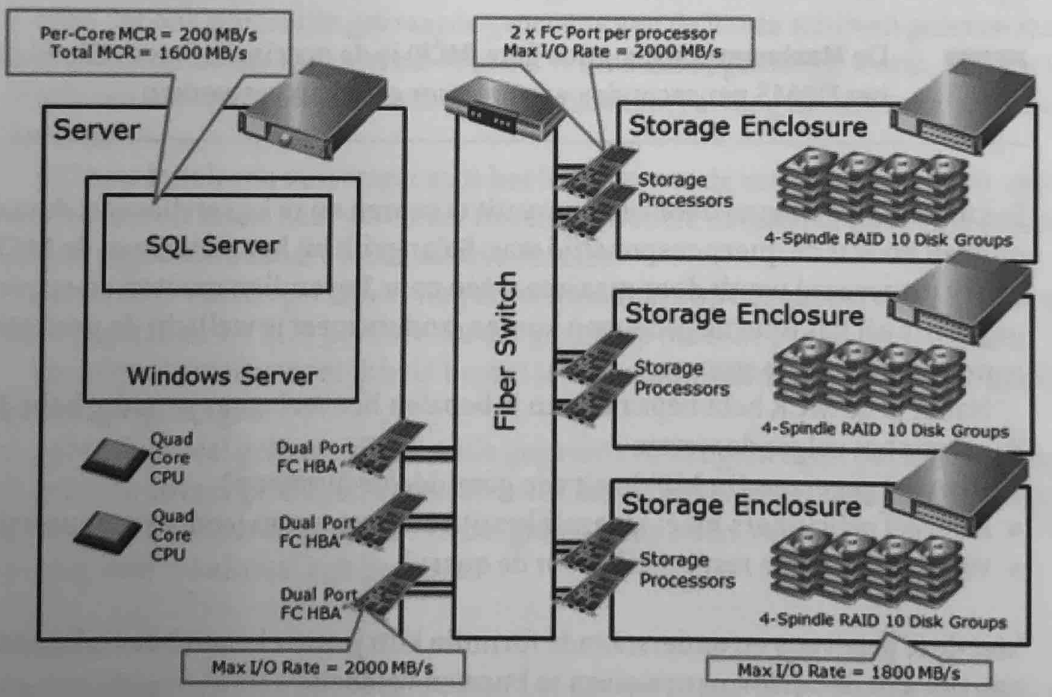
Veel leveranciers van zowel hardware als software kunnen op basis van dit soort classificeringen en ervaringen met andere klanten, aanbevelingen doen. Het is altijd goed te kijken naar ervaringen van anderen met overeenkomstige projecten. Al was het maar om niet het wiel opnieuw te hoeven uitvinden. Microsoft kent bijvoorbeeld de Fast Track Data Warehouse Architectures (*https://www.microsoft.com/en-us/sqlserver/solutions-technologies/data-warehousing/reference-architecture.aspx*). Hierin vind je aanbevelingen voor hard- en software gebaseerd op jouw classificering en Microsofts ervaringen met eerdere projecten.

Nu je enigszins een idee hebt van de grootte en complexiteit van het datawarehouse, gaan we in de volgende paragraaf kort in op hardware. In paragraaf 5.4 gaan we kijken hoe verschillende features van producten invloed hebben op de performance van het datawarehouse.

5.3 Hardware en software

Het gaat voor dit boek te ver uitgebreid in te gaan op benodigde en beschikbare hardware. In algemene termen zullen we in deze paragraaf de specificaties bezien die aan de hardware gesteld moeten worden.

Alle specificaties die in de vorige paragraaf zijn genoemd, moeten leiden tot inzicht in de benodigde hardware en de te kiezen architectuur. In figuur 5.4 zie je een overzicht van de hardwarecomponenten die een rol spelen. Het plaatje is afkomstig van de zojuist genoemde Fast Track Data Warehouse van Microsoft, vandaar dat je links in het plaatje SQL Server en Windows Server ziet staan. Het idee is echter ook geldig voor andere systemen.



Figuur 5.4 *Hardware-architecture*

Bij de Fast Track-architecturen wordt ervan uitgegaan dat alle componenten vergelijkbare specificaties moeten opleveren voor wat betreft de hoeveelheid gegevens die per seconde verwerkt kunnen worden. Als je bijvoorbeeld met een nieuw schijvenkabinet tweemaal zoveel bytes per seconde kunt lezen als met de oude harde schijven, maar de CPU of de netwerkkaarten kunnen niet meer data verwerken, heb je niets aan die verbeterde specificaties. Elke component kan de bottleneck zijn en de traagste zal dat zijn.

In figuur 5.4 is het systeem redelijk in balans. Het bestaat uit drie aparte storage enclosures in een SAN (Storage Area Network) met fiber channel tussen het SAN en de

server. Bij elke component staat de maximale datacapaciteit. Deze getallen moeten in orde van grootte hetzelfde zijn. Anders is er sprake van een bottleneck en overcapaciteit bij de andere componenten.

Een goede plek om te starten met het doorrekenen van een systeem is het berekenen van de capaciteit van de CPU. De overige getallen worden over het algemeen wel gepubliceerd door de verschillende hardwareleveranciers. Er zijn veel tools op de markt (bijvoorbeeld SQLIO voor SQL Server) die helpen de juiste getallen boven water te krijgen voor een specifiek systeem.

5.3.1 CPU-capaciteit

De Maximum Consumption Rate (MCR) is de maximale hoeveelheid gegevens die door één processorcore kan worden verwerkt. Met andere woorden, hoeveel MB per seconde kunnen maximaal verwerkt worden door één core? De MCR wordt bepaald door een combinatie van het soort processor, het moederbord van de server waarop die processor zich bevindt en het DBMS, SQL Server in het geval van figuur 5.4. Doordat het gaat over de specifieke combinatie van deze drie componenten, zul je de MCR altijd zelf moeten meten op de beoogde hardware met de juiste software.

De **Maximum Consumption Rate (MCR)** is de maximale hoeveelheid data die het DBMS per seconde per processor core kan verwerken.

Je kunt de MCR meten door een query uit te voeren en te kijken hoeveel data de query ophaalt en wat de query-responstijd was. Belangrijk bij het meten van de MCR is dat de query uitgevoerd wordt door maximaal één core. Bovendien moeten de opgevraagde gegevens uit het interne geheugen komen, anders meet je wellicht de performance van je netwerk of van je storage.

Nadat je de MCR hebt bepaald, kun je bepalen hoeveel cores je nodig hebt. Je moet daarvoor het volgende weten:

- Hoeveel gegevens (in MB) haalt een gemiddelde query op?
- Hoeveel gebruikers moet je tegelijkertijd ondersteunen (concurrent users)?
- Wat is de vereiste responstijd voor de query?

Met deze gegevens en onderstaande formule kun je uitrekenen hoeveel cores er nodig zijn om aan de performance-eisen te kunnen voldoen.

```
[Aantal cores] = ((grootte van resultset in MB / MCR) x aantal concurrent gebruikers) / responstijd in sec.
```

5.3.2 Overige hardwarespecificaties

Naast de CPU moet je nadenken over:

- Hoeveel geheugen heeft de datawarehouseserver nodig?
- Welke netwerkbandbreedte is nodig?
- Hoeveel opslagcapaciteit is nodig en welke lees- en schrijfsnelheid is nodig?

Geheugen

De benodigde hoeveelheid geheugen is moeilijk vast te stellen. Daar bestaat niet een mooie formule voor zoals hierboven beschreven voor de MCR. Gezien het feit dat gegevens ophalen uit het geheugen altijd veel sneller is dan gegevens ophalen van disks, zouden bij voorkeur alle gegevens in het geheugen moeten zitten. Er zijn ook steeds meer producten op de markt die daarop zijn gebaseerd.

Het op één na beste is om genoeg geheugen te hebben zodat de actieve dataset in het geheugen past. Het actieve deel van de gegevens is dat deel dat regelmatig geraadpleegd wordt. Stel dat 90% van de query's data ophaalt die betrekking heeft op het lopende jaar. Dan is dat jaar onderdeel van de actieve dataset. Eerdere jaargangen zijn daar geen onderdeel van.

De **actieve dataset** zijn de gegevens die regelmatig geraadpleegd worden en zich daarmee bij voorkeur in het geheugen van de server moeten bevinden.

Uit de informatieanalyse moet naar voren zijn gekomen welke vragen er regelmatig beantwoord moeten worden. Daaruit volgt de actieve dataset. Je moet er nog wel voor zorgen dat ook precies de gevraagde gegevens van de harde schijven gelezen kunnen worden. Zonder indexen zal voor elk record dat opgevraagd wordt, de hele tabel ingelezen moet worden. In dat laatste geval behoort alsnog de hele tabel tot de actieve dataset.

Er bestaat ook een vuistregel voor het bepalen van de actieve dataset. Uit onderzoek van Microsoft komt naar voren dat in een gemiddeld datawarehouse 20% van de opgeslagen gegevens behoort tot de actieve dataset.

Naast deze vuistregel zijn er nog ervaringscijfers op het gebied van hardware. Soms hoor je getallen als 4GB per core. Of dat genoeg is moet de praktijk uitwijzen.

In volgorde van wenselijkheid bepaal je met onderstaande manieren de hoeveelheid benodigd geheugen:

1. gebruik zoveel geheugen zodat alle gegevens in het geheugen passen. Anders,
2. gebruik zoveel geheugen dat de actieve dataset in het geheugen past. Anders,
3. zorg dat 20% van de gegevens in het geheugen passen. Anders,
4. zorg voor minimaal 4gb per core.

Netwerk

Het resultaat van elke query moet over het netwerk naar de gebruikersapplicatie gebracht worden. Als de resultaten groot zijn en/of als er heel veel query's worden afgevuurd, zal dat veel netwerkverkeer met zich meebrengen.

Uit de informatieanalyse komt naar voren welke rapporten hoe vaak gebruikt worden. In combinatie met de kennis hoe groot de dataset is die een rapport ophaalt, geeft dit informatie over de benodigde bandbreedte.

Opslag

Over opslag zou een heel boek vol geschreven kunnen worden (sterker nog, daar zijn hele boeken over volgeschreven). De open deur hier is dat je genoeg storage nodig hebt voor de hoeveelheid gegevens die je gaat opslaan. Vandaar een groot deel van de exer-

citie van paragraaf 5.2. Afhankelijk van het systeem dat je kiest, komt daar nog extra benodigde schijfruimte bij. SQL Server zal bijvoorbeeld bij grote query's gebruikmaken van tempdb, een systeemdatabase om tussenresultaten op te slaan. Ook tempdb heeft voldoende ruimte nodig.

Naast de hoeveelheid opslagcapaciteit is ook het soort opslagmedium belangrijk. Vanuit een performance-oogpunt is snelheid natuurlijk van belang. Wellicht dat solid state disks (SSD) een optie zijn. Deze disks zijn het duurste maar ook het snelste dat op dit moment op de markt is. SSD's zijn de enige permanente opslag die qua performance in de buurt komen van memory.

SAN's (Storage Area Networks) zijn prima geschikt voor datawarehousing. Voornamelijk het beheer, en het laten meegroeien van de opslagcapaciteit met het datawarehouse, zijn pluspunten.

5.3.3 Appliances

Deze paragraaf gaat er tot nu toe min of meer van uit dat er een server is met daar op een DBMS. Vaak is dat wat we in de praktijk doen, maar wel vooral bij kleine en middelgrote datawarehouses zoals beschreven in de tabel van paragraaf 5.2.6. Voor heel grote datawarehouses is het interessant om te kijken naar datawarehouse-appliances.

Een appliance is hardware en software ineen. Microsoft verkoopt APS (Analytics Platform System, voorheen PDW, wat staat voor Parallel Data Warehouse), in samenwerking met DELL en HP. SQL Server komt als het ware ingebakken in de hardware, volledig en optimaal geconfigureerd voor grote datawarehouses. Het functioneel ontwerp van het datawarehouse moet je uiteraard nog zelf maken. Je hoeft je niet druk te maken over zaken als file placement, RAID, parallellisme en andere configuraties. Dat soort zaken wordt geregeld door de appliance.

Een **datawarehouse appliance** is een geïntegreerde set van servers, opslagmedia, operating system, DBMS en andere software speciaal geïnstalleerd en geoptimaliseerd voor datawarehousing.

Uiteraard hebben ook de andere grote leveranciers vergelijkbare producten. Oracle heeft Exadata, IBM Netezza, en Teradata noemt het gewoon Teradata. Overal is het doel hetzelfde: voorgeconfigureerde hardware zodat je alleen nog maar met het functionele deel aan de slag hoeft. Met een beetje geluk is een hoofdstuk als dit in de toekomst helemaal overbodig.

Appliances hebben veel te bieden, maar zijn zeker niet goedkoop. Tussen zelf een gewone server helemaal configureren en inrichten voor het datawarehouse en een appliance zit nog wel een tussenweg, referenced architectures. Dit zijn specificaties opgesteld door hardwareleveranciers op basis van opgedane ervaringen bij klanten. Op basis van je eigen inschatting van grootte en de eisen ten aanzien van performance en beschikbaarheid, kunnen standaardoplossingen geboden worden.

Referenced architectures zijn vooraf gedefinieerde specificaties van hardware opgesteld met ervaring van vergelijkbare datawarehouseprojecten uit het verleden.

5.4 Performance features

Tot nu toe is in dit hoofdstuk vooral gekeken naar de hardware-aspecten van het inrichten van een datawarehouse. In algemene termen is gekeken naar benodigde hardware, voor een groot deel gebaseerd op hoe groot het datawarehouse wordt. Uiteraard is de workload, welke en wat voor query's en de drukte – hoeveel gebruikers – belangrijk. Ook de gekozen software, het DBMS, kan bijdragen aan een succesvolle implementatie. De meeste DBMS'en hebben overeenkomstige features.

Deze paragraaf is bedoeld om een meer concreet beeld te krijgen van wat een RDBMS kan. De voorbeelden zijn specifiek voor het product SQL Server van Microsoft en dus niet generiek. Daarom is deze paragraaf geen onderdeel van het gedrukte boek maar kun je hem online lezen op *www.leerboekbusinessintelligence.nl*.

De features waar we naar gaan kijken zijn:
- indexen;
- columnstore index;
- compressie;
- aggregatietabellen;
- partitionering.

5.5 Tot slot

In hoofdstuk 4 heb je geleerd wat voor soort tabelstructuren je kunt toepassen voor een datawarehouse. De daar geleerde theorie is onafhankelijk van het DBMS dat je gebruikt. In de praktijk is het noodzakelijk het functionele model aan te passen aan de gebruikte hard- en software en aan de te verwachte workload.

In dit vijfde hoofdstuk heb je geleerd welke extra aspecten van belang zijn om over na te denken voordat je een datawarehouse gaat implementeren.

Los van de hardware levert het DBMS features op die kunnen helpen om de juiste performance te halen. Voor het succesvol inzetten van deze features moet je de workload kennen. Je past het functionele databasedesign uit hoofdstuk 4 aan op het daadwerkelijke gebruik zoals dat uit de informatieanalyse is gekomen. Uiteraard zijn de beschikbare features afhankelijk van het gebruikte DBMS.